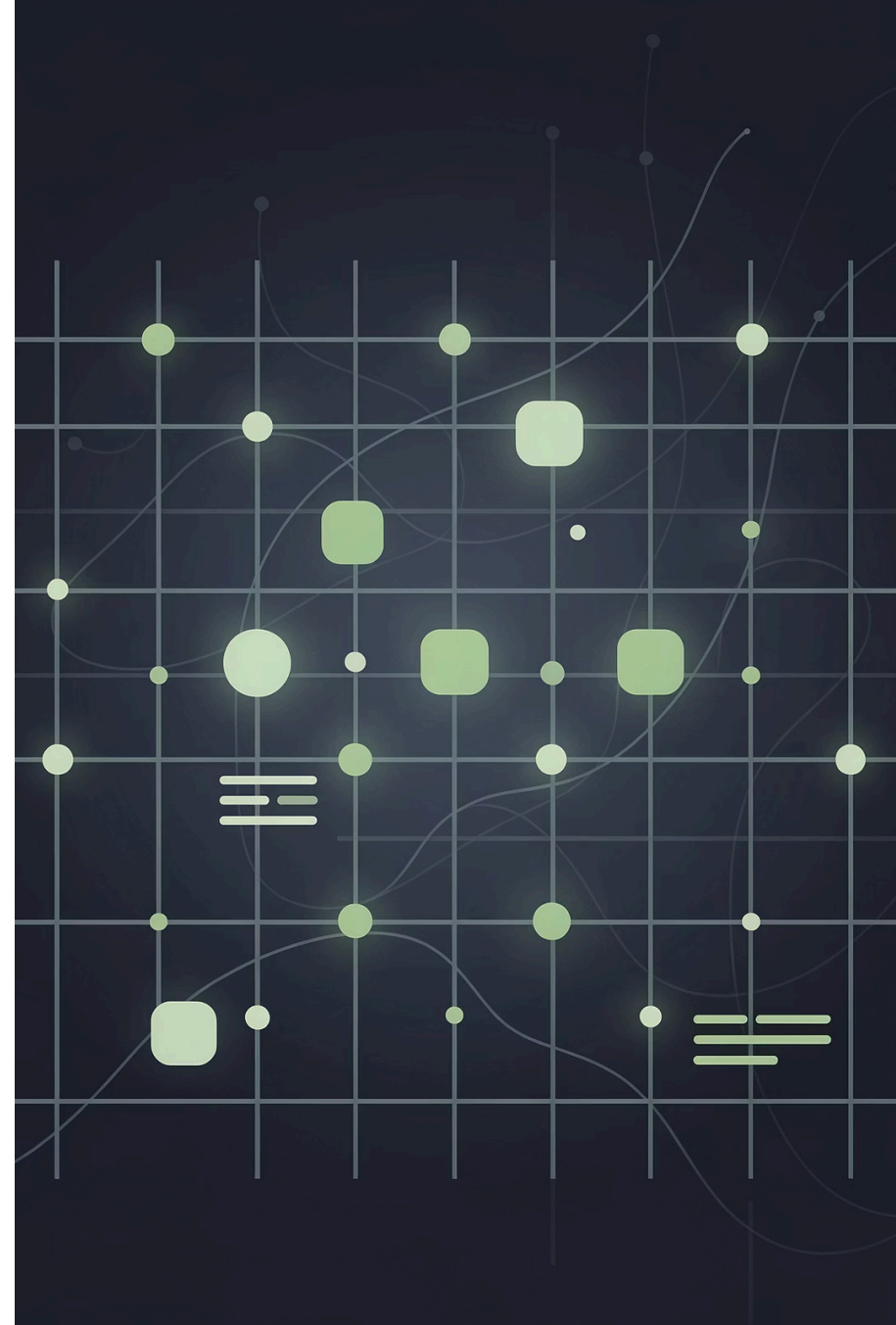


UNLOX

The Power of NumPy & Pandas

Your essential guide to scientific computing and data analysis in Python

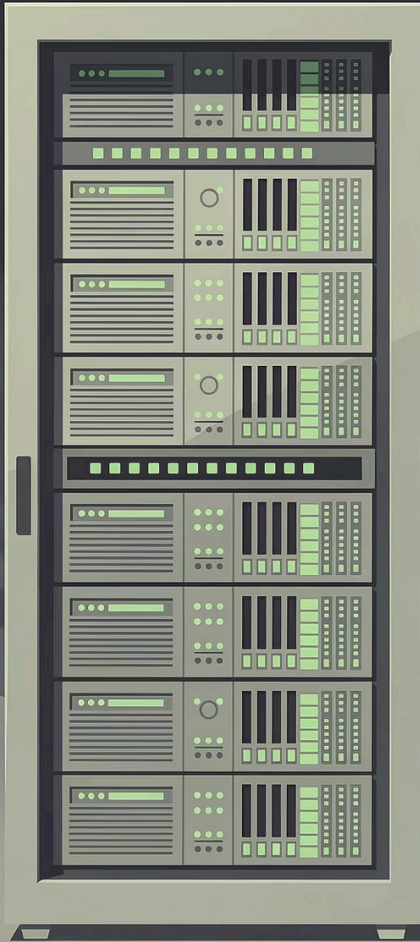


UNLOCK

CHAPTER 1

NumPy

The foundational library for scientific computing in Python



UNLOX

What Is NumPy?

NumPy — short for **Numerical Python** — is the foundational library that underpins virtually all scientific computing in Python. If you're working with numbers at scale, NumPy is where everything begins.

At its core, NumPy introduces the **ndarray** (n-dimensional array), a high-performance data structure purpose-built for numerical operations. Unlike standard Python lists, ndarrays store data in contiguous blocks of memory, enabling dramatically faster computation. This efficiency is what makes NumPy the cornerstone for libraries used across data science, machine learning, and AI.

UNLOX

Why NumPy Over Python Lists?

Standard Python lists are flexible but slow for number crunching. NumPy arrays are built for speed and simplicity. Here's how they compare:

Python Lists

- Store mixed data types
- Require explicit loops for math
- High memory overhead per element
- Slower for large-scale operations

NumPy Arrays

- Homogeneous data types for speed
- Vectorized operations — no loops needed
- Compact, contiguous memory layout
- Orders of magnitude faster at scale

UNLOCK

The ndarray: NumPy's Core

The **ndarray** is the heart of NumPy. It's a grid of values — all of the same type — indexed by a tuple of non-negative integers. Understanding its key properties will make everything else click.



Dimensions

Arrays can be 1D (vectors), 2D (matrices), or n-dimensional tensors for complex data.



Shape

The `.shape` attribute tells you the size along each dimension, e.g. `(3, 4)` for 3 rows and 4 columns.



Data Type

Every element shares the same dtype — `int32`, `float64`, etc. — enabling optimized memory use.

Vectorized Operations

One of NumPy's most powerful features is **vectorization** — performing element-wise operations on entire arrays without writing explicit for loops. This is faster, cleaner, and more Pythonic.

What You Can Do

- **Element-wise addition:** `a + b` adds corresponding elements
- **Element-wise multiplication:** `a * b` multiplies in parallel
- **Broadcasting:** `a + 5` adds a scalar to every element
- **Aggregation:** `a.mean()`, `a.sum()`, `a.std()` for quick stats

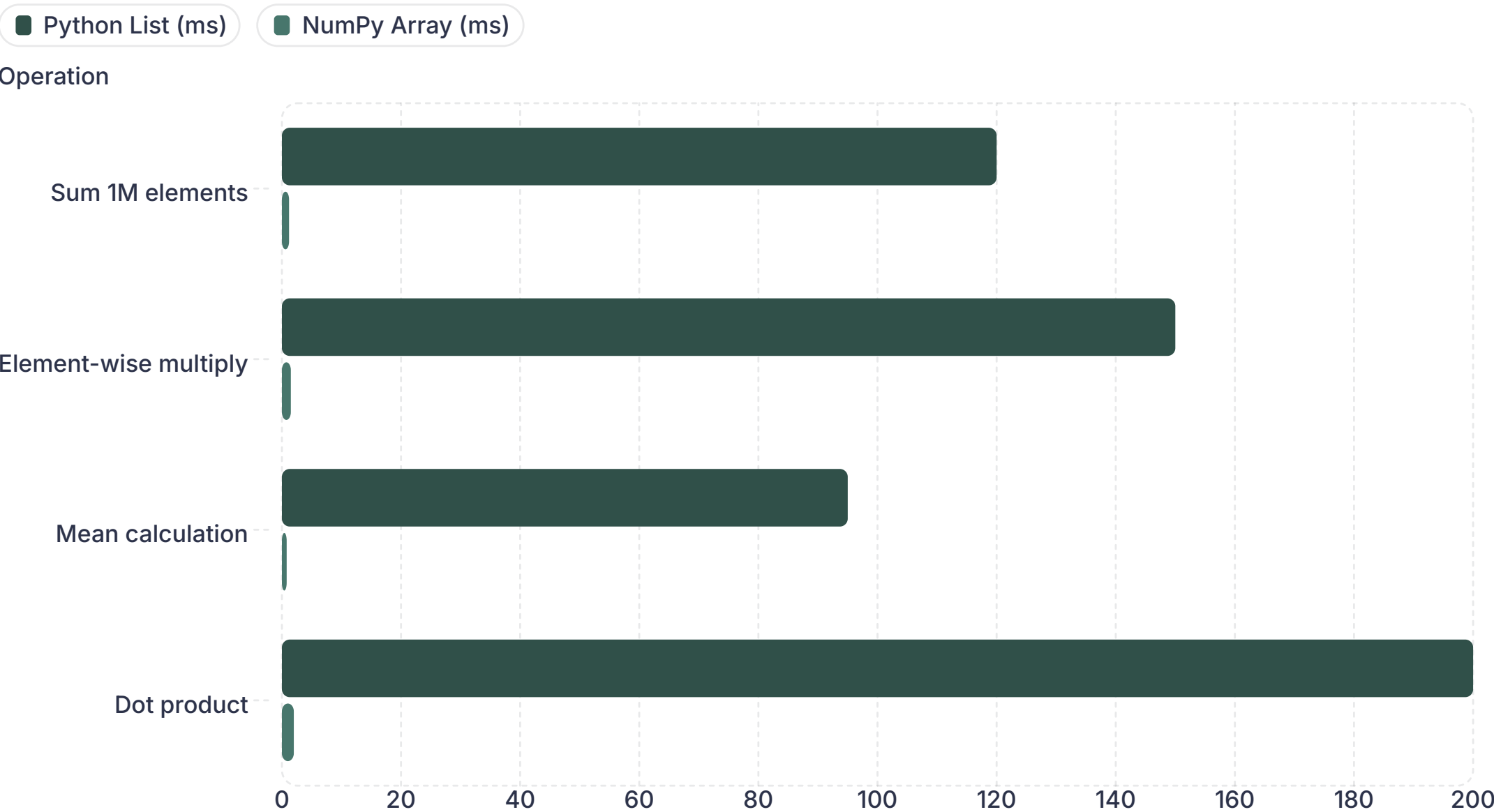
```
import numpy as np

a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

print(a + b) # [5, 7, 9]
print(a * b) # [4, 10, 18]
print(a + 10) # [11, 12, 13]
print(a.mean()) # 2.0
```

NumPy Performance at a Glance

The speed advantage of NumPy becomes strikingly clear as your dataset grows. Vectorized operations can be **10× to 100× faster** than equivalent Python loops.



These benchmarks illustrate typical performance on arrays with 1 million elements. NumPy's C-optimized backend makes the difference.

UNLOX

CHAPTER 2

Pandas

The ultimate toolkit for data manipulation and analysis

UNLOX

What Is Pandas?

While NumPy excels at raw numerical arrays, **Pandas** is purpose-built for **data manipulation and analysis**. It brings the intuition of a spreadsheet into the power of Python.

The primary data structure is the **DataFrame** — a two-dimensional, table-like object with labeled rows and columns. Think of it as a supercharged Excel sheet where every operation is scriptable, reproducible, and scalable. Pandas builds on top of NumPy, so you get all the speed benefits with a far richer interface for working with real-world, messy data.



The DataFrame: Your Data Workhorse

A DataFrame organizes data into rows and columns — each column can hold a different data type (integers, strings, floats, dates). Here's a quick look at what a typical DataFrame contains:

Name	Age	City	Score
Alice	29	New York	92.5
Bob	34	San Francisco	88.0
Carol	27	Chicago	95.3
David	41	Austin	79.8

Each row is a record, each column is a feature, and the index on the left gives every row a unique label. You can select, filter, group, and transform this data with just a few lines of code.

UNLOX

Create & Manipulate DataFrames

Pandas makes it simple to build DataFrames from scratch or reshape existing ones. Here are the most common ways to get started:

1

From a Dictionary

Pass a Python dict where keys become column names and values become column data: `pd.DataFrame({"Name": [...], "Age": [...]})`

2

From a List of Lists

Provide rows as nested lists and specify column names: `pd.DataFrame([[1, "A"], [2, "B"]], columns=["id", "label"])`

3

Manipulate on the Fly

Add columns, rename headers, filter rows, sort values, and chain operations together with expressive, readable syntax.

UNLOCK

Loading Data from Files & Databases

In real-world workflows, you rarely type data by hand. Pandas shines at ingesting data from external sources quickly and reliably.



CSV Files

The most common format. Use `pd.read_csv("data.csv")` to load comma-separated files into a DataFrame in one line.



Excel Files

Load `.xlsx` workbooks with `pd.read_excel("report.xlsx")`. You can even specify which sheet to import.

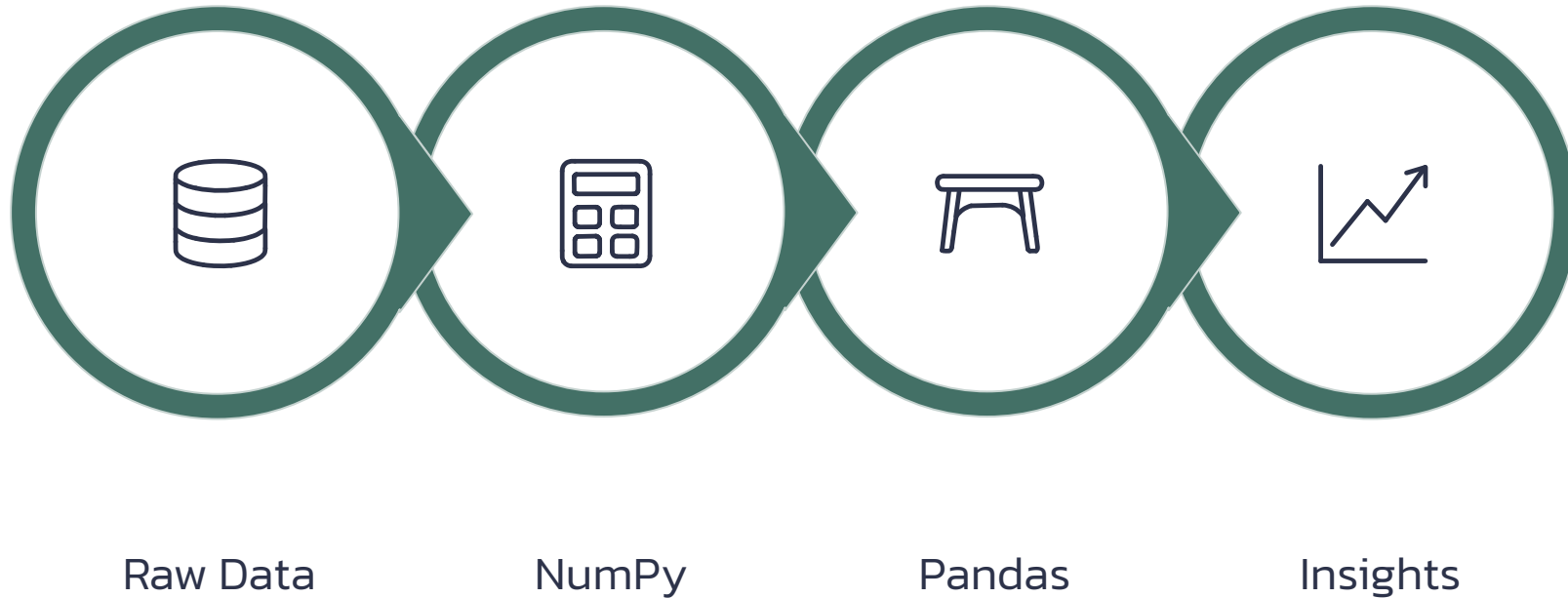


SQL Databases

Connect to relational databases using `pd.read_sql(query, connection)`. Supports PostgreSQL, MySQL, SQLite, and more.

NumPy + Pandas: Better Together

These two libraries are designed to complement each other. Understanding when to use which is the key to efficient data work.



NumPy handles the heavy numerical lifting under the hood, while Pandas provides the structured, labeled interface you interact with daily. Together, they form the backbone of the Python data science ecosystem.

Key Takeaways

- 1 NumPy is the foundation**
Its ndarray delivers blazing-fast vectorized operations that eliminate the need for slow Python loops.
- 2 Pandas is your analysis toolkit**
DataFrames give you a spreadsheet-like interface with the full power of Python for cleaning, exploring, and transforming data.
- 3 Load data from anywhere**
CSV, Excel, and SQL databases — Pandas makes ingestion effortless so you can focus on insights, not file parsing.
- 4 Practice vectorized thinking**
Embrace array-level operations over loops. This mindset is essential for scaling up to real-world datasets and AI workflows.

